Bölüm 14: NumPy Bölüm 2

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: NumPy_part2
7 Kasım 2026

İçindekiler

Bölüm 14 · NumPy Bölüm 2 3

Bölümün amacı 3

Konu anlatımı 3

Boolean indeksleme: maskeleme (Slayt 1–2) 3

Bileşik koşullar: &, |, ~ (Slayt 3) 3

Maskeyle değer atamak ve saymak (Slayt 4) 4

reshape ve -1 (Slayt 5–6) 4

ravel ve flatten (Slayt 7) 5

Transpoz (Slayt 8) 5

Birleştirme (Slayt 9) 5

Bölme (Slayt 10) 6

Ufunc'lar (Slayt 11–13) 6

Broadcasting (Slayt 14–16) 7

Toplama fonksiyonları ve axis (Slayt 17–19) 8

Sık yapılan hatalar 8

Kısa tekrar soruları 9

Cevaplar 9

Bölüm 14 · NumPy Bölüm 2

Bölümün amacı

Bu bölüm NumPy dizilerini veri analizine hazırlayan araçları öğretir. Sonunda şunları yapabilmelisin:

- Boolean maskeleme ile filtrelemek, `&`, `|`, `~` ile koşulları birleştirmek, maskeyle değer atamak ve saymak
- `reshape`, `ravel`, `flatten` ve transpoz ile şekil değiştirmek; view/copy farkını gözetmek
- `concatenate`, `vstack`, `hstack` ile birleştirmek ve `split` ile bölmek
- Ufunc'ları ve **broadcasting** kurallarını uygulamak
- Toplama fonksiyonlarını `axis` parametresiyle kullanmak

Konu anlatımı

Boolean indeksleme: maskeleme (Slayt 1–2)

Maskeleme, elemanları bir koşula göre seçmektir ve veri filtreleme/temizlemede temel bir tekniktir. İki adımlıdır:

1. **Maske oluştur:** vektörleştirilmiş bir karşılaştırma, her eleman için True/False içeren bir boolean dizi üretir.
2. **Maskeyi indeks olarak kullan:** True olan konumlardaki elemanlardan oluşan **yeni, 1 boyutlu** bir dizi döner.

PYTHON

```python
import numpy as np
data = np.array([3, 8, 2, 7, 1, 6, 0, 4])
mask = data > 4
print(mask)
print(data[mask])
```

ÇIKTI

```
[False  True False  True False  True False False]
[8 7 6]
```

Bileşik koşullar: &, |, ~ (Slayt 3)

`and`, `or`, `not` dizilerde çalışmaz; bunların yerine `&` (ve), `|` (veya), `~` (değil) kullanılır. Operatör önceliği yüzünden **her koşul parantez içine** alınmalıdır.

PYTHON

```python
import numpy as np
data = np.array([3, 8, 2, 7, 1, 6, 0, 4])
print(data[(data > 3) & (data < 7)])
print(data[(data == 5) | (data < 5)])
print(data[(data == 5) | ~(data < 5)])
```

ÇIKTI

```
[6 4]
[3 2 1 0 4]
[8 7 6]
```

Maskeyle değer atamak ve saymak (Slayt 4)

Maske seçilen elemanlara **değer atamak** için de kullanılır. `np.sum(maske)` True'ları 1 saydığı için koşulu sağlayan eleman **sayısını** verir.

PYTHON

```python
import numpy as np
new_data = np.arange(0, 20)
new_data[new_data % 2 != 0] = -1
print(new_data)
print('Negatif eleman sayısı:', np.sum(new_data < 0))
```

ÇIKTI

```
[ 0 -1  2 -1  4 -1  6 -1  8 -1 10 -1 12 -1 14 -1 16 -1 18 -1]
Negatif eleman sayısı: 10
```

reshape ve -1 (Slayt 5–6)

`reshape` veriyi değiştirmeden yeni bir şekil verir. Yeni şekil **aynı toplam eleman sayısını** içermelidir (değilse `ValueError`). Bellek düzeni izin verirse **view**, vermezse kopya döndürür. Bir boyut için **-1** yazılırsa NumPy onu kendisi hesaplar.

PYTHON

```python
import numpy as np
a = np.arange(12)
print(a.reshape(3, 4))
print(a.reshape(2, -1).shape, a.reshape(-1, 6).shape)
```

ÇIKTI

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 6) (2, 6)
```

ravel ve flatten (Slayt 7)

İkisi de çok boyutlu diziyi tek boyuta indirir. **ravel** mümkünse **view** döndürür (sonucu değiştirmek orijinali değiştirebilir); **flatten** her zaman **kopya** döndürür.

PYTHON

```python
import numpy as np
m = np.array([[1, 2], [3, 4]])
r = m.ravel()
f = m.flatten()
r[0] = 99
f[1] = 77
print(m)
print(r, f)
```

ÇIKTI

```
[[99  2]
 [ 3  4]]
[99  2  3  4] [ 1 77  3  4]
```

Transpoz (Slayt 8)

Transpoz satırlarla sütunları yer değiştirir: `.T` ya da `np.transpose()`. Yüksek boyutlarda `np.transpose(t, axes=(...))` eksenlerin yeni sırasını alır: (d1, d2, d3) şekli `axes=(2, 0, 1)` ile **(d3, d1, d2)** olur.

PYTHON

```python
import numpy as np
m = np.arange(6).reshape(2, 3)
print(m.T)
t = np.zeros((2, 3, 4))
print(np.transpose(t, axes=(2, 0, 1)).shape)
```

ÇIKTI

```
[[0 3]
 [1 4]
 [2 5]]
(4, 2, 3)
```

Birleştirme (Slayt 9)

- `np.concatenate((a, b), axis=…)`: genel amaçlı; diziler birleştirme ekseni **dışındaki** boyutlarda aynı şekle sahip olmalıdır.
- `np.vstack`: üst üste (satır ekler) = `axis=0`
- `np.hstack`: yan yana (sütun ekler) = `axis=1`

PYTHON

```python
import numpy as np
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
c = np.array([[7], [8]])
print(np.concatenate((a, b), axis=0))
print(np.vstack((a, b)).shape)
print(np.hstack((a, c)))
```

ÇIKTI

```
[[1 2]
 [3 4]
 [5 6]]
(3, 2)
[[1 2 7]
 [3 4 8]]
```

Bölme (Slayt 10)

`np.split(dizi, belirtec, axis)` birleştirmenin tersidir. Belirteç bir tam sayı **N** ise dizi N **eşit** parçaya bölünür (bölünemezse `ValueError`); bir indeks listesi ise bu indeksler **bölme noktalarıdır**.

PYTHON

```python
import numpy as np
x = np.arange(9)
print(np.split(x, 3))
print(np.split(x, [2, 5]))
```

ÇIKTI

```
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
[array([0, 1]), array([2, 3, 4]), array([5, 6, 7, 8])]
```

Ufunc'lar (Slayt 11–13)

Ufunc (universal function) dizilerde **eleman eleman** çalışan, döngü gerektirmeyen vektörleştirilmiş fonksiyondur. Aritmetik operatörler ufunc'lara bağlıdır: `a + b` içeride `np.add(a, b)`'yi çağırır.

Kategori	Örnekler	
Aritmetik	`np.add`, `np.subtract`, `np.multiply`, `np.divide`, `np.power`	
Matematiksel	`np.sqrt`, `np.exp`, `np.log`, `np.abs`	

Kategori	Örnekler	
Trigonometrik	`np.sin`, `np.cos`, `np.tan`, `np.arcsin` (radyan)	
Karşılaştırma	`np.greater`, `np.less`, `np.equal` → **boolean dizi**	
Mantıksal/bit	`np.logical_and`, `np.bitwise_or` (`	`), np.bitwise_not (~`)

PYTHON
```
import numpy as np
a = np.array([1, 4, 9])
b = np.array([1, 2, 3])
print(np.add(a, b), a + b, np.multiply(a, b))
print(np.sqrt(a), np.greater(a, 3))
```

ÇIKTI
```
[ 2  6 12] [ 2  6 12] [ 1  8 27]
[1. 2. 3.] [False  True  True]
```

Broadcasting (Slayt 14–16)

Broadcasting, farklı şekillerdeki diziler arasında işlem yapmayı sağlar: küçük dizi büyüğün şekline "esnetilir". Esnetme bellekte **gerçek bir kopya oluşturmaz**, bu yüzden verimlidir. En basit hali dizi ile skaler arasındaki işlemdir.

Şekiller **en sağdaki boyuttan başlayarak** karşılaştırılır:

1. Boyut sayısı farklıysa az boyutlu olanın şekli **soldan 1'lerle** doldurulur: (4,) → (1, 4).
2. Bir boyutta biri **1** ise o dizi diğerinin boyutuna esnetilir.
3. Boyutlar farklı ve **hiçbiri 1 değilse** `ValueError`.

PYTHON
```
import numpy as np
print(np.array([1, 2, 3]) * 10)
matrix = np.ones((3, 4))
vector = np.arange(4)
print(matrix + vector)
print((np.arange(3).reshape(3, 1) * np.arange(4)).shape)
```

ÇIKTI
```
[10 20 30]
[[1. 2. 3. 4.]
 [1. 2. 3. 4.]
 [1. 2. 3. 4.]]
(3, 4)
```

PYTHON
```python
import numpy as np
np.ones((3, 4)) + np.arange(3)
```

ÇIKTI
```
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
```

Toplama fonksiyonları ve axis (Slayt 17–19)

`np.sum`, `np.mean`, `np.std`, `np.min`/`np.amin`, `np.max`/`np.amax`, `np.prod` varsayılan olarak **tüm dizi** üzerinde çalışır ve **tek bir skaler** döndürür. `axis` parametresi işlemin yönünü belirler:

- `axis=0`: satırlar çökertilir → **her sütun için** bir sonuç
- `axis=1`: sütunlar çökertilir → **her satır için** bir sonuç

PYTHON
```python
import numpy as np
m = np.array([[1, 2, 3], [4, 5, 6]])
print(np.sum(m), np.mean(m), np.min(m), np.max(m), np.prod(m), round(np.std(m),
↪  3))
print(m.sum(axis=0), m.sum(axis=1), np.max(m, axis=1))
```

ÇIKTI
```
21 3.5 1 6 720 1.708
[5 7 9] [ 6 15] [3 6]
```

Bölümün kaynakları (Slayt 19): Boolean masking (Medium), NumPy quickstart, NumPy Array Manipulation (Medium), Universal functions (NumPy v2.3), Python Data Science Handbook – Broadcasting, Broadcasting (NumPy v2.3).

Sık yapılan hatalar

- **Maskede `and` / `or` kullanmak** → `ValueError: The truth value of an array ... is ambiguous`.

PYTHON
```python
import numpy as np
data = np.array([3, 8, 2, 7])
data[data > 3 and data < 8]
```

ÇIKTI
```
ValueError: The truth value of an array with more than one element is
↪ ambiguous. Use a.any() or a.all()
```

- **Bileşik koşulda parantezi unutmak:** `data > 3 & data < 7` önce `3 & data` hesaplar.
- **Eşit bölünmeyen diziyi `split` ile N parçaya bölmek** → `ValueError`.
- `vstack`**'te sütun sayıları,** `hstack`**'te satır sayıları farklı diziler** → `ValueError`.
- `ravel` **sonucunun bağımsız olduğunu sanmak:** view olabilir; bağımsız kopya için `flatten` ya da `.copy()`.
- `axis=0`**'ı "satır başına" sanmak:** `axis=0` her **sütun** için sonuç verir.

Kısa tekrar soruları

1 `data = np.array([5, 12, 7, 20])` için `data[(data > 6) & (data < 15)]` nedir?

2 `np.arange(24).reshape(4, -1).shape` nedir?

3 ravel ile flatten arasındaki fark nedir?

4 `np.zeros((2, 3))` ile `np.ones((5, 3))` `vstack` edilirse şekil ne olur? `hstack` edilirse?

5 (2, 3) ile (3,) şekilli diziler toplanabilir mi? (2, 3) ile (2,) peki?

6 `m = np.array([[1, 2], [3, 4]])` için `m.max(axis=0)` ve `m.sum(axis=1)` nedir?

Cevaplar

1 `[12 7]`

2 `(4, 6)`

3 ravel mümkünse view döndürür (orijinal değişebilir); flatten her zaman kopya döndürür.

4 `vstack` → `(7, 3)`; `hstack` → `ValueError` (satır sayıları 2 ≠ 5).

5 (2, 3) + (3,) olur: (3,) → (1, 3) → (2, 3). (2, 3) + (2,) olmaz: sondaki boyutlar 3 ve 2, hiçbiri 1 değil → `ValueError`.

6 `[3 4]` ve `[3 7]`.